

TP 2 - Spring Data JPA

Este segundo trabajo practico consistirá en proveer un nuevo servicio de persistencia utilizando Spring Data JPA en lugar de usar Hibernate directamente, como se hizo en el primer trabajo práctico.

Para lograr esto, deberá crearse una nueva implementación de la interfaz `ToursService` llamada `ToursServiceImpl`, tomando como base la implementación anterior, que deberá utilizar repositorios de Spring Data para resolver cualquier acceso a datos requerido. En este caso se debe crear un repositorio por clase a persistir. Los repositorios implementados con Spring Data no son clases sino interfaces, y deberán extender `CrudRepository` definiendo los argumentos necesarios: tipo de entidad y tipo de id. Por ejemplo:

```
public interface RouteRepository extends CrudRepository, Long>
```

Las consultas se pueden declarar utilizando query methods, los cuales serán implementados automáticamente por Spring Data (por ejemplo: `findBy<Propiedad>`), o mediante HQL utilizando la annotation `@Query` (notar que, dado que se trata de una interfaz, los métodos no pueden tener implementación).

Las clases de tests, `ToursApplicationTests` y `ToursQuerysTests` validan, al igual que en la etapa previa, distintas implementaciones del servicio para que pueda probar libremente estas.

Para configurar SpringData se utiliza la clase `SpringDataConfiguration`, que deberán editar con sus credenciales de acceso a la BD.

La infraestructura del proyecto provista se encuentra en el mismo repositorio del TP previo, en la rama **tp2-springdata**:

En concreto, se han modificado: la clase de configuración `AppConfig`, se ha reemplazado `HibernateConfiguration` con la clase de configuración `SpringDataConfiguration` y se han actualizado las dependencias necesarias en `pom.xml`; `DBInitializer` y por supuesto, los archivos de Testing.

Sección 1 - Introducción a Spring Data JPA

1.1 Spring Data JPA y su lugar en el ecosistema

1. ¿Qué es Spring Data JPA? ¿Qué problema resuelve respecto de usar Hibernate directamente? Describir dos situaciones del proyecto donde Spring Data JPA simplifica código que en la Práctica 1 requería implementación manual.

Spring Data JPA es una **capa de abstracción superior** que se sitúa por encima del ORM (Hibernate) dentro del ecosistema Spring. Su objetivo es simplificar significativamente la implementación de la capa de acceso a datos (repositorios), permitiendo que el desarrollador se enfoque en definir interfaces en lugar de escribir clases de implementación concretas.

Los problemas que resuelve respecto de usar Hibernate directamente son:

- **Reducción de Código Boilerplate:** En Hibernate directo (Práctica 1), el desarrollador debe crear clases de implementación, inyectar la `SessionFactory`, gestionar sesiones con `getCurrentSession()` y escribir manualmente métodos para cada operación básica (guardar, buscar, borrar). Spring Data JPA elimina esta repetición implementando automáticamente estas operaciones en tiempo de ejecución.
- **Abstracción del Repositorio:** Permite tratar la persistencia como una simple colección de objetos mediante interfaces, desacoplando la lógica de negocio de los detalles técnicos de la `Session` de Hibernate.
- **Generación Automática de Consultas:** Evita tener que escribir HQL/JPQL para búsquedas simples, ya que puede derivar la consulta directamente del nombre del método definido en la interfaz.

Las situaciones donde Spring Data JPA simplifica el código serían:

- **Las operaciones CRUD básicas** ya que Spring provee todas las implementaciones de estas operaciones sin nosotros tener que escribir código.
- **Consultas de búsqueda simples** ya que al usar la técnica de Query Methods, el framework se vuelve capaz de interpretar el nombre del método, generar el HQL correspondiente y ejecutarlo automáticamente.

2. Spring Data JPA no es un ORM sino una capa de abstracción sobre el ORM. Explicar la diferencia: ¿Qué hace Spring Data JPA? ¿Qué sigue haciendo Hibernate internamente?

Diferencia: Capa de Abstracción vs. Motor ORM

Característica	Spring Data JPA (Abstracción)	Hibernate (Motor ORM)
Rol Principal	Provee una capa para implementar repositorios de forma declarativa.	Realiza el mapeo físico entre objetos Java y tablas relacionales.
Implementación	Genera la implementación de los repositorios en tiempo de ejecución mediante proxies dinámicos.	Es la implementación concreta de la especificación JPA que habla con la DB.
Consultas	Deriva consultas automáticamente a partir del nombre de los métodos (Query Methods).	Traduce el HQL/JPQL o las consultas derivadas a SQL nativo

Característica	Spring Data JPA (Abstracción)	Hibernate (Motor ORM)
		según el dialecto de la base de datos.
Gestión Técnica	Maneja la propagación de transacciones con <code>@Transactional</code> .	Gestiona el Persistence Context (Caché de Nivel 1) y el <i>Dirty Checking</i> .

¿Qué hace Spring Data JPA? Su función principal es eliminar el **código boilerplate** (repetitivo) que antes se escribía a mano en los repositorios.

- **Repositorios basados en Interfaces:** Permite definir interfaces que extienden de `CrudRepository` sin necesidad de escribir la clase de implementación.
- **Generación de Consultas:** Interpreta palabras clave como `findBy`, `existsBy` o `countBy` para crear la lógica de búsqueda sin escribir código SQL ni HQL.
- **Soporte de Paginación y Ordenamiento:** Provee soporte nativo mediante las interfaces `Pageable` y `Sort`.
- **Reducción de Acoplamiento:** Oculta los detalles técnicos de la `Session` de Hibernate, permitiendo que el programador piense en términos de colecciones de objetos.

¿Qué sigue haciendo Hibernate internamente? Aunque Spring Data JPA maneja las interfaces, Hibernate sigue operando en las capas inferiores realizando las tareas críticas:

- **Mapeo Objeto-Relacional:** Lee las anotaciones (`@Entity`, `@Table`, `@Column`) y determina cómo transformar los atributos en columnas.
- **Gestión del Ciclo de Vida:** Controla los estados de las entidades (**Transient**, **Managed**, **Detached**, **Removed**).
- **Manejo de Sesiones y Caché:** Gestiona la conexión física con la base de datos a través de JDBC y mantiene la caché de Nivel 1 para optimizar lecturas.
- **Sincronización (Flush):** Detecta cambios en los objetos gestionados y decide cuándo disparar las sentencias `INSERT`, `UPDATE` o `DELETE` para mantener la base de datos sincronizada.
- **Carga Perezosa (Lazy Loading):** Implementa el patrón **Proxy** para cargar asociaciones solo cuando se necesitan, evitando el uso excesivo de memoria.

3. La siguiente tabla lista tareas relacionadas con la persistencia. Marcar con una X la columna de la tecnología que resuelve ese problema en la nueva implementación con Spring Data JPA:

Tarea	JDBC	Hibernate	Spring Data JPA
Abrir y cerrar la conexión a la base de datos	X		

Tarea	JDBC	Hibernate	Spring Data JPA
Implementar save(), findById() y deleteById()			X
Gestionar el ciclo de vida de las entidades (@Entity)		X	
Derivar una consulta a partir del nombre del método			X
Manejar el pool de conexiones		X	
Propagar transacciones con @Transactional			X
Generar la implementación del repositorio en runtime			X
Mapear ResultSet a objetos Java		X	
Proveer soporte nativo de paginación (Pageable)			X

1.2 Configuración del Proyecto

4. Listar los cambios que deben realizarse en el proyecto de la Práctica 1 para incorporar Spring Data JPA.

Cambios que se deberían de hacer

Elemento	Antes	Después
<code>pom.xml</code>	<code>hibernate-core</code> + <code>spring-orm</code>	<code>spring-boot-starter-data-jpa</code>
<code>HibernateConfiguration.java</code>	Configuración manual	Se elimina
<code>application.properties</code>	Propiedades de Hibernate	Propiedades de Spring JPA
Repositorios	Interfaces + Impls con <code>SessionFactory</code>	Solo interfaces que extienden <code>JpaRepository</code>
<code>AppConfig</code>	Creación manual de beans	Vacío
<code>ToursServiceImpl</code>	Sin cambios de lógica	Solo ajuste en métodos con <code>Pageable</code>

5. En la Práctica 1 se configuraba Hibernate mediante una clase de configuración Java para obtener la `SessionFactory`. ¿Cómo se reemplaza esta configuración usando `application.properties` en Spring Boot?

Forma de reemplazar la obtención manual de la `SessionFactory` con el uso de `application.properties` :

- **Centralización de la Configuración:** Antes había que tener un archivo `HibernateConfiguration.java` donde se instanciaba y configuraba manualmente el Bean

de `SessionFactory`, esto ahora se reemplaza por el archivo `application.properties`, que permite definir todos los parámetros de persistencia de forma declarativa sin escribir código Java de infraestructura.

- **Configuración del origen de datos (DataSource):** Los parámetros de conexión que antes se pasaban al objeto de configuración de Hibernate ahora se definen con el prefijo `spring.datasource`:
 - **spring.datasource.url:** Reemplaza la URL de conexión a la base de datos (ej. `jdbc:mysql://localhost:3306/tours`).
 - **spring.datasource.username** y **password:** Reemplazan las credenciales de acceso.
 - **spring.datasource.driver-class-name:** Define el driver JDBC a utilizar.
- **Configuración específica de Hibernate y JPA:** Para indicarle a Hibernate cómo comportarse, se utilizan propiedades con el prefijo `spring.jpa`:
 - **Dialecto:** Se utiliza `spring.jpa.properties.hibernate.dialect` para especificar el lenguaje SQL del motor (ej. `MySQL8Dialect`).
 - **Gestión del esquema:** La propiedad `spring.jpa.hibernate.ddl-auto` reemplaza la configuración manual de creación de tablas. Sus valores posibles incluyen `create`, `update`, `validate` o `none`.
 - **Visualización de SQL:** Se puede habilitar la consola con `spring.jpa.show-sql=true`.
- **Gestión interna de la infraestructura:** Antes se tenía que gestionar la `SessionFactory` para obtener sesiones, en Spring Boot **el framework gestiona automáticamente** la creación del `EntityManagerFactory` (el equivalente JPA a la `SessionFactory`) basándose en estas propiedades. Ya no necesita inyectar ni manipular la `SessionFactory` manualmente en los repositorios, ya que Spring Data JPA se encarga de implementar los repositorios en tiempo de ejecución utilizando esta configuración de fondo.

```
spring.datasource.url=jdbc:mysql://localhost:3306/bd2_tours?
createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=create
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.use_sql_comments=false
```

6. Describir las propiedades más relevantes de Spring Data JPA que deben configurarse en `application.properties` para este proyecto. Incluir al menos: `datasource` (`url`, `username`, `password`, `driver`), `dialecto` de Hibernate, y la propiedad que controla si Hibernate debe

crear, actualizar o validar el esquema. ¿Cuál de estos valores conviene usar durante el desarrollo?

Propiedades más relevantes para el proyecto:

- *Configuración del Origen de Datos (DataSource)*: Estas propiedades establecen la conexión física con el motor de base de datos:
 - **spring.datasource.url**: Define la ubicación de la base de datos (protocolo, host, puerto y nombre de la base). Ejemplo: `jdbc:mysql://localhost:3306/bd2_tours`.
 - **spring.datasource.username**: El usuario del motor (típicamente `root`).
 - **spring.datasource.password**: La contraseña de acceso.
 - **spring.datasource.driver-class-name**: El driver JDBC específico para MySQL (`com.mysql.cj.jdbc.Driver`).
- *Dialecto de Hibernate*:
 - **spring.jpa.database-platform** (o `hibernate.dialect`): Indica a Hibernate qué variante de SQL debe generar. Dado que SQL es un estándar con variaciones según el motor, el dialecto asegura que el ORM use la sintaxis correcta para MySQL (ej. `org.hibernate.dialect.MySQLDialect`).
- *Gestión del Esquema (DDL-Auto)*: La propiedad **spring.jpa.hibernate.ddl-auto** controla cómo Hibernate interactúa con la estructura de tablas al iniciar la aplicación. Sus valores principales son:
 - **create**: Borra las tablas existentes y las crea de nuevo al arrancar (El que creo que sería lo más óptimo para el proyecto).
 - **update**: Intenta modificar las tablas existentes para que coincidan con las entidades `@Entity`, manteniendo los datos.
 - **validate**: Solo comprueba si las tablas existen y coinciden con el modelo; si no, lanza un error.
 - **none**: No realiza ninguna acción.

1.3 La Interfaz CrudRepository y la jerarquía de repositorios

7. ¿Qué es CrudRepository? ¿De qué interfaz hereda y qué operaciones provee automáticamente?

CrudRepository es una interfaz genérica que provee una **capa de abstracción** sobre el ORM (Hibernate) para realizar operaciones de creación, lectura, actualización y borrado (CRUD) sobre una entidad específica. Su principal ventaja es que permite al desarrollador definir interfaces de repositorio sin necesidad de escribir la implementación lógica (código boilerplate), ya que Spring genera el código necesario en tiempo de ejecución.

`CrudRepository<T, ID>` hereda directamente de la interfaz **Repository<T, ID>**.

- **Repository** es una interfaz "marcador" (marker interface) que no posee métodos propios, pero sirve para que el framework identifique a la interfaz como un repositorio de Spring Data y capture los parámetros de tipo (la clase de la entidad y el tipo de su clave primaria)

Operaciones que provee automáticamente: Al extender `CrudRepository`, el repositorio incorpora automáticamente un conjunto de métodos listos para usar, evitando la necesidad de interactuar manualmente con la `Session` de Hibernate para tareas comunes:

- **save(S entity):** Guarda una entidad. Spring Data JPA decide automáticamente si debe realizar un `INSERT` (si es nueva) o un `UPDATE` (si ya existe en la base de datos).
- **findById(ID id):** Recupera una entidad por su identificador único.
- **existsById(ID id):** Retorna un booleano indicando si existe un registro con ese ID.
- **findAll():** Devuelve todas las instancias de la entidad almacenadas en la base de datos.
- **count():** Retorna la cantidad total de entidades de ese tipo.
- **deleteById(ID id):** Elimina el registro correspondiente al identificador provisto.
- **delete(T entity):** Elimina una instancia específica de la entidad.
- **deleteAll():** Elimina todos los registros gestionados por el repositorio.

8. ¿Qué agrega cada nivel de la jerarquía respecto del anterior? Describir brevemente las diferencias entre `CrudRepository`, `PagingAndSortingRepository` y `JpaRepository`, indicando que operaciones o capacidades incorpora cada uno.

Diferencias y capacidades de cada nivel:

- **CrudRepository:** Es la interfaz base para las operaciones fundamentales de persistencia.
 - **Hereda de:** `Repository<T, ID>` (una interfaz marcadora sin métodos).
 - **Capacidades:** Provee las operaciones básicas de creación, lectura, actualización y borrado (**CRUD**).
 - **Operaciones principales:** `save()`, `findById()`, `findAll()`, `deleteById()`, `count()` y `existsById()`.
 - **Finalidad:** Resolver el código repetitivo (*boilerplate*) de persistencia simple sin necesidad de implementar clases concretas.
- **PagingAndSortingRepository:** Este nivel agrega herramientas para gestionar grandes volúmenes de datos de forma eficiente a través de la interfaz.
 - **Hereda de:** `CrudRepository<T, ID>`.
 - **Capacidades:** Incorpora soporte nativo para **paginación y ordenamiento** de los resultados.
 - **Operaciones principales:**
 - `findAll(Sort sort)`: Permite obtener todas las entidades ordenadas por uno o varios atributos.

- `findAll(Pageable pageable)` : Permite obtener un subconjunto de los datos (una "página") especificando el número de página y el tamaño.
- **Finalidad:** Mejorar el rendimiento y la experiencia de usuario al no cargar la base de datos completa en memoria, devolviendo objetos de tipo `Page<T>` o `Slice<T>`.
- **JpaRepository:** Es la interfaz más completa y está orientada específicamente a tecnologías JPA (como Hibernate).
 - **Hereda de:** `PagingAndSortingRepository<T, ID>`.
 - **Capacidades:** Agrega métodos específicos para el control del **Persistence Context** de JPA y operaciones por lotes (*batch*).
 - **Operaciones principales:**
 - `flush()` : Sincroniza los cambios pendientes en memoria inmediatamente con la base de datos.
 - `saveAndFlush()` : Guarda la entidad y ejecuta el flush en un solo paso.
 - `deleteInBatch()` : Elimina múltiples entidades en una sola consulta SQL, lo cual es más eficiente que el borrado individual.
 - **Mejora de retorno:** Sobrescribe métodos como `findAll()` para que devuelvan una `List<T>` en lugar de un `Iterable<T>`, facilitando el uso de la API de colecciones de Java.
 - **Finalidad:** Brindar un control técnico más fino sobre la sesión de Hibernate y optimizar ejecuciones masivas.
- 9. Crear las interfaces de repositorio para las entidades del modelo extendiendo `CrudRepository`. Indicar los parámetros de tipo correctos (entidad e ID) para cada una: `PurchaseRepository`, `RouteRepository`, `UserRepository`, `ServiceRepository`, `SupplierRepository` y `ReviewRepository`.

```
package unlp.info.bd2.repositories;

import org.springframework.data.repository.CrudRepository;
import unlp.info.bd2.model.Purchase;

import java.util.List;
import java.util.Optional;
import java.util.Date;

public interface PurchaseRepository extends CrudRepository<Purchase, Long> {

    Optional<Purchase> findByCode(String code);

    List<Purchase> findByUserUsername(String username);
}
```

```
        long countByDateBetween(Date start, Date end);
    }
}
```

```
package unlp.info.bd2.repositories;

import java.util.List;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.data.repository.query.Param;
import org.springframework.data.domain.Pageable;

import unlp.info.bd2.model.Route;
import unlp.info.bd2.model.Stop;

public interface RouteRepository extends CrudRepository<Route, Long> {

    List<Route> findByPriceLessThan(float price);

    @Query("SELECT COUNT(p) FROM Purchase p WHERE p.route.id = :routeId")
    long countPurchasesByRoute(@Param("routeId") Long routeId);

    List<Route> findByStopsContaining(Stop stop);

    @Query("SELECT MAX(SIZE(r.stops)) FROM Route r")
    Long findMaxStopOfRoutes();

    @Query("""
        SELECT r FROM Route r
        WHERE r NOT IN (
            SELECT DISTINCT p.route FROM Purchase p
        )
    """)
    List<Route> findRoutesNotSell();

    @Query("""
        SELECT p.route FROM Purchase p
        WHERE p.review IS NOT NULL
        GROUP BY p.route
        ORDER BY AVG(p.review.rating) DESC
    """)
    List<Route> findTop3RoutesWithMaxRating(Pageable pageable);

    @Query("SELECT COUNT(r) FROM Route r JOIN r.tourGuideList g WHERE g.id = :id")
}
```

```
    long countRoutesByGuide(@Param("id") Long guideId);  
}
```

```
package unlp.info.bd2.repositories;  
  
import java.util.List;  
import java.util.Optional;  
  
import org.springframework.data.jpa.repository.Query;  
import org.springframework.data.repository.CrudRepository;  
import org.springframework.data.repository.query.Param;  
  
import unlp.info.bd2.model.TourGuideUser;  
import unlp.info.bd2.model.User;  
  
public interface UserRepository extends CrudRepository<User, Long> {  
  
    Optional<User> findByUsername(String username);  
  
    @Query("""  
        SELECT DISTINCT p.user FROM Purchase p  
        WHERE p.totalPrice ≥ :amount  
        """)  
    List<User> findUsersSpendingMoreThan(@Param("amount") float amount);  
  
    @Query("""  
        SELECT DISTINCT g FROM Route r  
        JOIN r.tourGuides g, Purchase p  
        WHERE p.route = r AND p.review.rating = 1  
        """)  
    List<TourGuideUser> findTourGuidesWithRating1();  
}
```

```
package unlp.info.bd2.repositories;  
  
import java.util.List;  
import java.util.Optional;  
  
import org.springframework.data.domain.Pageable;  
import org.springframework.data.jpa.repository.Query;  
import org.springframework.data.repository.CrudRepository;  
  
import unlp.info.bd2.model.Service;
```

```

public interface ServiceRepository extends CrudRepository<Service, Long> {

    Optional<Service> findByNameAndSupplierId(String name, Long supplierId);

    @Query("""
        SELECT item.service FROM ItemService item
        GROUP BY item.service
        ORDER BY SUM(item.quantity) DESC
        """)
    List<Service> findMostDemandedService(Pageable pageable);
}

```

```

package unlp.info.bd2.repositories;

import java.util.List;
import java.util.Optional;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.data.domain.Pageable;

import unlp.info.bd2.model.Supplier;

public interface SupplierRepository extends CrudRepository<Supplier, Long> {

    Optional<Supplier> findByAuthorizationNumber(String
authorizationNumber);

    @Query("""
        SELECT s.supplier FROM Service s
        JOIN s.itemServiceList item
        JOIN item.purchase p
        GROUP BY s.supplier
        ORDER BY COUNT(p) DESC
        """)
    List<Supplier> findTopSuppliersInPurchases(Pageable pageable);
}

```

```

package unlp.info.bd2.repositories;

import org.springframework.data.repository.CrudRepository;

import unlp.info.bd2.model.Review;

```

```
public interface ReviewRepository extends CrudRepository<Review, Long> {  
  
}
```

10. ¿Cómo genera Spring Data JPA la implementación concreta de los repositorios en tiempo de ejecución? ¿Qué rol juega el proxy dinámico de Java en este mecanismo?

Generación de la Implementación en Tiempo de Ejecución

- **Detección:** Al arrancar la aplicación, Spring escanea el proyecto en busca de interfaces que hereden de la jerarquía de `Repository`.
- **Instanciación Automática:** Spring Data JPA genera automáticamente una implementación concreta para estas interfaces en **tiempo de ejecución** (runtime).
- **Inyección:** Una vez generada esta implementación, Spring la registra como un *bean* en el contenedor de dependencias, permitiendo que la capa de servicio la reciba e utilice como cualquier otro objeto.

Rol del Proxy Dinámico

- **Intercepción de Mensajes:** Dado que una interfaz no tiene código ejecutable, el Proxy actúa como un **intermediario o "máscara"**. Cuando un servicio invoca un método del repositorio (ej. `findById` o un `findByEmail` personalizado), la llamada es capturada por el Proxy en lugar de buscar una implementación física en el código del alumno.
 - **Delegación de Lógica:** El Proxy analiza el mensaje recibido:
 - Si es un método estándar de `CrudRepository`, delega la ejecución a una implementación interna de Spring que maneja la `Session` de Hibernate.
 - Si es un **Query Method** (ej. `findByUsername`), el Proxy activa el motor de Spring Data que deriva la consulta HQL a partir del nombre del método y la ejecuta.
 - **Gestión de Transacciones:** El Proxy también se encarga de "envolver" la llamada con comportamientos adicionales, como la apertura y cierre de la sesión de base de datos o la propagación de transacciones si el método está marcado con `@Transactional`.
11. ¿Qué diferencia hay entre `save()` en Spring Data JPA y `session.save()` / `session.merge()` en Hibernate directo? ¿Cómo decide Spring Data JPA si debe hacer un INSERT o un UPDATE?

Diferencia entre `save()` (Spring Data) y métodos de Hibernate: En **Hibernate directo**, se debe elegir el método según el estado de la entidad:

- **`session.save()`:** Se utiliza específicamente para objetos nuevos (**transient**), asignándoles un identificador y haciéndolos persistentes.
- **`session.merge()`:** Se utiliza principalmente para objetos **detached** (desasociados). Copia el estado del objeto sobre una instancia gestionada en la sesión actual; si el objeto no existe,

lo crea; si existe, lo actualiza.

Spring Data JPA save() simplifica esto al proveer un **único punto de entrada**. No nos preocupamos por el estado técnico del objeto (si es nuevo o desasociado), ya que el framework delega la lógica interna a JPA/Hibernate de manera transparente.

¿Cómo decide Spring Data JPA entre INSERT o UPDATE? Spring Data JPA utiliza una lógica interna para determinar si una entidad es "nueva" o ya existe en la base de datos. Esta decisión se basa principalmente en la **inspección del identificador (@Id) de la entidad**:

1. **Si el ID es null (o 0 en tipos primitivos):**

- Spring Data JPA interpreta que el objeto es **nuevo**.
- Internamente, invoca la operación de **persistencia** (equivalente a un `INSERT` en SQL).

2. **Si el ID tiene un valor:**

- Spring Data JPA asume que el objeto **ya existe** en la base de datos (o es un objeto desasociado que debe sincronizarse).
- Internamente, invoca la operación de **unión** (`merge`), lo que generalmente resulta en una sentencia `UPDATE` de SQL tras el **flush** de la sesión.

Sección 2 - Repositorios con Spring Data JPA

2.1 Migración de Repositorios

12. Comparar el código de `PurchaseRepository` de la Práctica 1 (con `Session` de Hibernate) con el nuevo `PurchaseRepository` de Spring Data JPA. ¿Cuántas líneas de código se eliminaron? ¿Qué operaciones ya no es necesario implementar manualmente?

Comparando los 2 códigos de los repositorios podemos ver:

- Se eliminaron 60 líneas de código aproximadamente.
 - Antes se tenía que implementar el código para las operaciones `save()`, `findById()`, `findAll()` y `delete()`, ahora ya se incluyen solas en el `CrudRepository`. También se elimina el manejo de Hibernate con la `SessionFactory`, y la necesidad de escribir consultas HQL manuales con `createQuery` y `setParameter`.
13. En la Práctica 1, los repositorios gestionaban internamente la `Session`. ¿Que recibe ahora un servicio que necesita usar un repositorio Spring Data? ¿Cómo se declara esa dependencia? ¿Dónde queda ahora la lógica de apertura y cierre de sesiones?

El servicio ahora recibe una **interfaz de repositorio Spring Data**, ya que Spring automáticamente genera la implementación en tiempo de ejecución.

La **dependencia** se puede declarar de 2 formas:

- Utilizando `@Autowired`.

```
@Autowired
private PurchaseRepository purchaseRepository;
```

- Haciendo una inyección por constructor (La mejor práctica).

```
private final PurchaseRepository purchaseRepository;

public ToursServiceImpl(PurchaseRepository purchaseRepository) {
    this.purchaseRepository = purchaseRepository;
}
```

La lógica de apertura y cierre de sesiones queda a cargo de **Spring y Hibernate**, gestionada automáticamente mediante `@Transactional` y el `EntityManager` **interno**.

2.2 Query Methods

14. ¿Cómo funciona la generación de consultas por nombre de método en Spring Data JPA?
- Describir las palabras clave principales que puede interpretar (`findBy`, `existsBy`, `countBy`, `deleteBy`) y cómo se combinan con los atributos de la entidad.

Funcionamiento de los Query Methods: Este mecanismo permite que Spring Data JPA **derive automáticamente** la lógica de una consulta (usualmente traducida a HQL/JPQL e internamente a SQL) analizando el nombre de un método definido en la interfaz de un repositorio.

- **Interpretación:** El framework analiza el nombre del método, identifica un prefijo (palabra clave) y busca que el resto del nombre coincida con los nombres de los **atributos de la entidad** gestionada.
- **Asociación de Parámetros:** Spring Data JPA asocia cada parámetro definido en el método con la condición correspondiente en la consulta, respetando estrictamente el **orden de aparición** de los atributos en el nombre del método.
- **Generación en Runtime:** La implementación física de esta consulta se genera en tiempo de ejecución a través de proxies dinámicos, por lo que el desarrollador no necesita escribir la sentencia de base de datos manualmente.

Palabras clave principales

- **findBy** (o `readBy`, `getBy`, `queryBy`): Indica una operación de **recuperación** (SELECT). Retorna una instancia de la entidad o una colección según la firma del método.

- **existsBy**: Genera una consulta que retorna un **booleano** (`true` o `false`). Se utiliza para verificar la existencia de un registro sin necesidad de cargar todos sus datos en memoria.
- **countBy**: Ejecuta una consulta de **agregación** que devuelve un valor numérico (`Long` o `int`) representando la cantidad total de registros que cumplen el criterio.
- **deleteBy** (o `removeBy`): Ejecuta una operación de **eliminación** física de los registros que coincidan con los criterios de búsqueda especificados.

Combinación con Atributos y Operadores: Estas palabras clave se combinan con los atributos de la entidad (siguiendo la convención *CamelCase*) y pueden incluir operadores lógicos y de comparación:

- **Operadores Lógicos:** Se pueden encadenar atributos usando **And** u **Or**.
 - *Ejemplo:* `findByNameAndPrice` .
- **Criterios de Comparación:** Permite usar sufijos como **LessThan**, **Between**, **IsNull**, o **Like**.
- **Ordenamiento:** Se puede agregar ordenamiento estático al nombre usando **OrderBy**.
 - *Ejemplo:* `findAllOrderByPriceAsc` .

15. ¿Cómo se pasan los parámetros a un Query Method? ¿Cómo hace Spring Data JPA para asociar cada parámetro del método con la condición correspondiente en la consulta? ¿Qué ocurre si el orden de los parámetros no coincide con el orden de las condiciones en el nombre del método?

Los parámetros se pasan como **argumentos estándar en la firma del método** de Java. Spring Data JPA analiza estos argumentos y los utiliza para completar los valores de las condiciones de búsqueda que generó a partir del nombre del método.

La asociación se realiza de forma **estrictamente posicional**. Spring Data JPA recorre el nombre del método de izquierda a derecha para identificar los atributos de la entidad y, en ese mismo orden, va asignando los parámetros declarados en el método.

- Si el método es `findByNameAndPrice(String name, Double price)`, el framework asocia:
 1. El primer atributo del nombre (`Name`) con el primer parámetro del método (`name`).
 2. El segundo atributo del nombre (`Price`) con el segundo parámetro del método (`price`).

Si el orden de los parámetros en la firma del método no respeta el orden de aparición de los atributos en el nombre del mismo, se producirá un **error de asociación**, con las siguientes consecuencias:

- **Error de Tipos:** Si los tipos de datos son incompatibles (por ejemplo, intentar asignar un `String` a una columna numérica), la aplicación lanzará una excepción en tiempo de ejecución al intentar ejecutar la consulta.

- **Resultados Incorrectos (Error Lógico):** Si los tipos son compatibles pero el orden es erróneo, la consulta se ejecutará pero buscará valores en las columnas equivocadas, devolviendo resultados incorrectos para la lógica de negocio sin que el programador reciba un aviso de error técnico.
16. Investigar y para cada uno de los siguientes requerimientos indicar si es posible resolverlo con un Query Method y escribir la firma del método correspondiente. Si no es posible, explicar por qué:
1. Buscar todas las Purchase de un usuario dado su username.
 2. Verificar si existe alguna Purchase para una Route dada.
 3. Contar cuantas Review tienen un rating mayor o igual a un valor dado.
 4. Obtener todas las Route cuyo precio sea menor a un valor dado, ordenadas por nombre.
 5. Buscar un User por su email.
 6. Obtener los top 3 Route con mayor cantidad de Purchase.

Buscar todas las Purchase de un usuario dado su username.

```
List<Purchase> findByUsername(String username);
```

Verificar si existe alguna Purchase para una Route dada.

```
boolean existsByRoute(Route route);
```

Contar cuantas Review tienen un rating mayor o igual a un valor dado.

```
long countByRatingGreaterThanOrEqualTo(int rating);
```

Obtener todas las Route cuyo precio sea menor a un valor dado, ordenadas por nombre.

```
List<Route> findByPriceLessThanOrderByNameAsc(double price);
```

Buscar un User por su email.

```
Optional<User> findByEmail(String email);
```

Obtener los top 3 Route con mayor cantidad de Purchase.

- Esta no se puede hacer porque ya es muy compleja, requiere de:
 - Agrupación (GROUP BY)

- Conteo (`COUNT`)
- Orden por agregación (`ORDER BY COUNT( ... ) DESC`)
- Limit (`TOP 3`)
 - Aunque existe la palabra clave `Top3` , los Query Methods están limitados a filtrar por atributos existentes en la entidad. Para resolver este requerimiento se necesita realizar una **agregación** (`COUNT`) de la entidad relacionada (`Purchase`) y un **agrupamiento** (`GROUP BY`), capacidades que exceden la derivación por nombre de método.

17. Investigar cuales son las palabras clave (keywords) disponibles en Spring Data JPA para construir Query Methods. ¿Qué tipos de condiciones, comparaciones y operadores lógicos soporta?

Categorías de palabras clave y operadores soportados

1. **Prefijos de Intención:** Determinan el tipo de operación que se realizará sobre la base de datos:
 - **findBy, readBy, getBy, queryBy:** Inician una consulta de selección (SELECT).
 - **existsBy:** Retorna un booleano indicando si el registro existe.
 - **countBy:** Retorna un valor numérico con la cantidad de registros que cumplen el criterio.
 - **deleteBy, removeBy:** Ejecutan una operación de eliminación física.
2. **Operadores Lógicos:** Permiten combinar múltiples condiciones en una sola consulta:
 - **And:** Ambas condiciones deben cumplirse (ej. `findByNameAndPrice`).
 - **Or:** Al menos una de las condiciones debe cumplirse (ej. `findByEmailOrUsername`).
3. **Operadores de Comparación:** Se utilizan para filtrar los valores de los atributos:
 - **Igualdad:** Por defecto (si no se especifica palabra clave) o usando **Is, Equals**.
 - **Desigualdad:** **Not**.
 - **Rangos:** **Between** (requiere dos parámetros: inicio y fin).
 - **Menor/Mayor que:** **LessThan, LessThanEqual, GreaterThan, GreaterThanEqual**.
 - **Nulidad:** **IsNull, IsNotNull** (o **NotNull**).
4. **Operadores para Cadenas de Texto (Strings):** Especializados en la búsqueda parcial o de patrones:
 - **Like, NotLike:** Búsqueda por patrón (el desarrollador debe incluir los caracteres `%`).
 - **StartingWith:** Busca registros que comiencen con el texto dado (agrega `%` al final automáticamente).
 - **EndingWith:** Busca registros que terminen con el texto dado (agrega `%` al inicio automáticamente).
 - **Containing:** Busca coincidencias en cualquier parte de la cadena (agrega `%` al inicio y final).

5. **Ordenamiento y Limitación:** Permiten organizar los resultados devueltos:
- **OrderBy:** Seguido del nombre del atributo y la dirección (ej. `findAllByOrderByPriceAsc`).
 - **Asc / Desc:** Para definir orden ascendente o descendente respectivamente.
 - **First / Top:** Se pueden usar tras el prefijo para limitar la cantidad de resultados (ej. `findTop3ByOrderByPriceDesc`).
6. ¿Qué limitaciones tienen los Query Methods? Describir al menos tres casos del modelo donde esta técnica resulta insuficiente y es necesario otro enfoque. Describa los otros enfoques posibles para implementar dichas consultas.

Limitaciones de los Query Method:

- **Complejidad y Legibilidad:** A medida que se agregan más condiciones (usando `And` , `Or`), el nombre del método se vuelve excesivamente largo y difícil de leer, lo que compromete el mantenimiento del código.
- **Capacidad de Expresión Limitada:** No permiten realizar operaciones de **agregación complejas** (como `GROUP BY` o `HAVING`), ni manejar **subconsultas** de forma nativa en el nombre del método.
- **JOINS No Explícitos:** Aunque permiten navegar por asociaciones simples (ej. `findByUserUsername`), los Query Methods resultan insuficientes para **JOINS complejos** que involucran múltiples niveles o condiciones específicas de unión que no se deducen de la jerarquía de propiedades.
- **Naturaleza Estática:** Están definidos en tiempo de compilación por el nombre; no permiten construir criterios de búsqueda de forma dinámica según parámetros que pueden o no estar presentes en tiempo de ejecución.

Casos del modelo donde resultan insuficientes:

- **Obtener los top 3 Route con mayor cantidad de Purchase:**
 - **Por qué falla:** Requiere realizar un conteo (`COUNT`) de una entidad relacionada, agrupar los resultados por la entidad principal (`GROUP BY`) y luego ordenar por ese valor calculado. Los Query Methods solo pueden filtrar u ordenar por atributos existentes, no por resultados de funciones de agregación sobre relaciones.
- **Obtener el servicio más demandado (getMostDemandedService):**
 - **Por qué falla:** Al igual que el anterior, implica contar cuántas veces aparece cada `Service` en la lista de `ItemService` de todas las `Purchase` , agrupar y seleccionar el máximo. Esta lógica de agregación y selección de un único registro basado en un cálculo no es derivable del nombre.
- **Obtener las rutas con mayor rating promedio (getTop3RoutesWithMaxRating):**
 - **Por qué falla:** Requiere navegar de `Route` a `Purchase` y de ahí a `Review` , calcular el promedio (`AVG`) de los ratings y agrupar por ruta. La profundidad de la relación y el

cálculo del promedio exceden la capacidad de interpretación semántica del framework.

2.3 Consultas con @Query

19. En la Práctica 1 se utilizaron consultas HQL (Hibernate Query Language). ¿Que diferencia hay entre HQL y JPQL (Java Persistence Query Language)? ¿Son intercambiables? ¿Cuál de los dos acepta @Query por defecto?

HQL (Hibernate Query Language) vs. JPQL (Java Persistence Query Language)

- **JPQL** es el lenguaje de consulta definido por la **especificación estándar** de JPA (Java Persistence API). Su objetivo es ser una sintaxis común e independiente de la implementación utilizada.
- **HQL** es el lenguaje de consulta propio de **Hibernate**. Dado que Hibernate existía antes de que se creara el estándar JPA, HQL sirvió como base para el diseño de JPQL.
- **Diferencia de capacidades:** HQL es un **superconjunto** de JPQL. Esto significa que HQL contiene todas las capacidades de JPQL pero agrega funciones y características adicionales específicas de Hibernate que aún no han sido incorporadas o estandarizadas en JPA.

No son del todo intercambiables, aunque existe un alto grado de compatibilidad:

- **De JPQL a HQL:** Una consulta escrita en JPQL estándar siempre será válida en Hibernate, ya que este motor entiende el estándar por completo.
- **De HQL a JPQL:** No siempre son intercambiables. Si una consulta utiliza características específicas de Hibernate (como ciertas funciones de fecha, subconsultas en lugares específicos o palabras clave no estandarizadas), esa consulta **no será válida** en otros proveedores de JPA que no sean Hibernate.

En Spring Data JPA, la anotación **@Query** acepta JPQL por defecto.

- Cuando el desarrollador escribe una consulta dentro de `@Query("...")`, el framework asume que se trata de **JPQL** y la procesa como tal a través del motor de persistencia (Hibernate).
- Si se desea utilizar SQL nativo en lugar de JPQL/HQL, se debe especificar explícitamente el atributo `nativeQuery = true` dentro de la anotación.

20. ¿Qué diferencia hay entre una consulta @Query con JPQL y una con nativeQuery = true? ¿Cuándo conviene cada una? Dar un ejemplo concreto del modelo para cada caso.

La diferencia fundamental entre una consulta con **JPQL** y una con **SQL nativo** (`nativeQuery = true`) dentro de la anotación `@Query` radica en el **sujeto de la consulta** y su **portabilidad**.

Característica	JPQL (Java Persistence Query Language)	SQL Nativo (<code>nativeQuery = true</code>)
Sujeto	Opera sobre el modelo de objetos (clases y atributos).	Opera sobre el modelo relacional (tablas y columnas).
Portabilidad	Es independiente del motor de BD. Hibernate traduce el JPQL al dialecto específico (MySQL, Oracle, etc.).	Está atado estrictamente al motor de base de datos utilizado en el proyecto.
Semántica	Entiende conceptos de objetos como herencia, polimorfismo y navegación de asociaciones (<code>p.user.name</code>).	Desconoce la lógica de objetos; requiere <code>JOINS</code> explícitos y nombres físicos de tablas.
Flexibilidad	Limitado a lo que el estándar JPA o el proveedor (Hibernate) soporta.	Permite usar funciones propietarias del motor (ej. funciones de fecha de MySQL)

Conviene JPQL (Default):

- En la gran mayoría de los casos de negocio.
- Cuando se quiere mantener el código limpio y desacoplado de la base de datos física.
- Cuando la consulta implica navegar relaciones simples o jerarquías de clases (ej. buscar en `User` y que incluya `DriverUser`).

Conviene SQL Nativo:

- Cuando se necesita realizar **optimizaciones de performance extremas** que el ORM no logra generar eficientemente.
- Para utilizar **funciones específicas** del motor (como cálculos espaciales o de texto completo) que no están en el estándar JPQL.

21. ¿Cómo se pasan parámetros a una consulta `@Query`? Describir y comparar las dos formas: parámetros posicionales (`?1`, `?2`) y parámetros nombrados (`@Param`). ¿Cuál es la forma recomendada y por qué?

Formas de pasar parámetros en `@Query`

1. **Parámetros Posicionales (`?1` , `?2` , ...):** En esta modalidad, los parámetros en la consulta se identifican por un signo de interrogación seguido de un índice numérico que representa su posición en la firma del método.
 - **Sintaxis:** `@Query("SELECT u FROM User u WHERE u.email = ?1 AND u.name = ?2")`
 - **Funcionamiento:** El primer argumento del método se asigna a `?1` , el segundo a `?2` , y así sucesivamente.
 - **Riesgo:** Es altamente sensible al orden. Si se cambia la firma del método o se agregan nuevos filtros en la consulta sin actualizar los números, se producirán errores de tipos o

resultados incorrectos.

2. **Parámetros Nombrados** (`:nombre` y `@Param`): Esta forma utiliza nombres explícitos tanto en la consulta como en la firma del método para realizar la vinculación.
 - **Sintaxis en consulta:** Se utiliza el prefijo de dos puntos (`:`) seguido del nombre (ej. `:email`).
 - **Sintaxis en método:** Se utiliza la anotación `@Param("nombre")` antes de cada argumento.
 - **Funcionamiento:** Spring Data JPA busca la coincidencia exacta entre el nombre definido en `@Param` y el nombre con prefijo `:` en la sentencia, independientemente del orden en que aparezcan en el método.

La forma recomendada es el uso de **Parámetros Nombrados** (`@Param`). Razones:

1. **Robustez ante cambios:** Al no depender del orden posicional, el desarrollador puede refactorizar la firma del método (agregar o mover parámetros) sin riesgo de que la consulta empiece a buscar datos en columnas equivocadas o lance excepciones de tipo en tiempo de ejecución.
2. **Claridad del código:** En consultas complejas con múltiples filtros, el uso de nombres (como `:username` , `:minPrice`) hace que la sentencia sea mucho más fácil de leer y depurar que una serie de números anónimos.
3. **Prevención de Errores Lógicos:** Minimiza la posibilidad de intercambiar valores accidentalmente (por ejemplo, pasar un `id` donde iba un `precio`) si ambos son del mismo tipo, ya que el nombre actúa como un contrato explícito de vinculación.

2.4 Paginación y Ordenamiento

22. ¿Qué es la interfaz Pageable? ¿Cómo se construye una instancia de Pageable? ¿Qué información encapsula (número de página, tamaño, ordenamiento)?

Es una interfaz abstracta que define el contrato para solicitar **subconjuntos de datos (páginas) a un repositorio**. Al incluir un parámetro de tipo `Pageable` en un método de repositorio, Spring Data JPA intercepta la llamada y aplica automáticamente las cláusulas de limitación (`LIMIT` y `OFFSET`) y ordenamiento correspondientes en la base de datos.

Esta interfaz encapsula tres parámetros críticos para la recuperación parcial de información:

- **Número de página (Page Number):** El índice de la página que se desea recuperar (comenzando generalmente en 0).
- **Tamaño de la página (Page Size):** La cantidad máxima de registros que deben devolverse en esa página específica.
- **Ordenamiento (Sort):** Los criterios de ordenación (atributo y dirección, como ascendente o descendente) que deben aplicarse antes de segmentar los datos.

Aunque `Pageable` es una interfaz, el framework provee la implementación `PageRequest` para ser utilizada en la capa de servicio.

Típicamente, se construye utilizando métodos estáticos de fábrica. Por ejemplo, para obtener la **segunda página** de resultados (índice 1), con un tamaño de **10 registros** y ordenados por **fecha descendente**, se invocaría de la siguiente manera desde un servicio:

```
Pageable pageable = PageRequest.of(1, 10, Sort.by("date").descending());
```

23. ¿Qué diferencia hay entre los tipos de retorno `Page` y `Slice`? ¿Cuándo conviene cada uno?
¿Qué consulta adicional ejecuta `Page` que `Slice` no ejecuta?

Diferencia entre los tipos de retorno `Page` y `Slice`

- `Page<T>` : Es un contenedor que posee información completa sobre la paginación. Sabe cuántos elementos totales hay en la base de datos, cuántas páginas existen en total y en qué página se encuentra el usuario actualmente.
- `Slice<T>` : Es una versión más "ligera". Solo sabe si existe una página siguiente disponible, pero desconoce el número total de registros y el total de páginas.

La diferencia técnica fundamental es que `Page<T>` **ejecuta una consulta de conteo adicional** (`SELECT COUNT(*) ...`) después de recuperar los datos de la página solicitada. Esta consulta es necesaria para calcular el total de páginas disponibles.

Por el contrario, `Slice<T>` **no ejecuta esta consulta de conteo**. Para saber si hay una página siguiente, Spring Data solicita internamente un elemento extra (si pides 10, trae 11); si recibe ese registro adicional, marca que hay una página siguiente disponible y luego descarta el registro extra antes de devolver el resultado al servicio.

Conviene usar `Page<T>` :

- Cuando la interfaz de usuario requiere un **navegador de páginas clásico** (ej. "Página 1 de 50" o botones para saltar a una página específica).
- Cuando el volumen de datos es moderado y el costo de la consulta `COUNT` es despreciable.

Conviene usar `Slice<T>` :

- Cuando se implementan interfaces de **"scroll infinito"** o botones de **"Cargar más"**, donde al usuario solo le interesa saber si hay más contenido, no cuántas páginas faltan.
- **Por rendimiento**: En tablas con millones de registros, la consulta `COUNT` puede ser extremadamente lenta y costosa para la base de datos. `Slice` evita este problema por completo, optimizando la respuesta del servidor.

24. Mostrar como se invocará desde la capa de servicio un método paginado para obtener la segunda página de compras de un usuario, con 10 resultados ordenados por fecha descendente.

Requerimiento de ejemplo: Nos piden la segunda página, con tamaño 10 de las compras de un usuario, tiene que estar ordenado de forma descendente por la fecha.

- *En el servicio:*

```
Pageable pageable = PageRequest.of(
    1,
    10,
    Sort.by("date").descending()
);

Page<Purchase> purchases = purchaseRepository.findByUserUsername(username,
pageable);
```

- *En el repositorio:*

```
Page<Purchase> findByUserUsername(String username, Pageable pageable);
```

25. ¿Cómo se agrega ordenamiento a un Query Method sin usar Pageable? ¿Qué palabra clave se usa en el nombre del método? Mostrar un ejemplo concreto con el modelo.

El **ordenamiento** sin usar `Pageable` se le puede agregar a un Query Method directamente en el nombre usando la palabra clave `OrderBy`. Ejemplo concreto:

```
List<Purchase> findByUserUsernameOrderByDateDesc(String username);
```

2.5 Implementación de las consultas del modelo

26. Para cada consulta, indicar la estrategia elegida (Query Method o @Query) e implementarla en el repositorio correspondiente:

Consulta	Repositorio	Estrategia elegida
<code>getAllPurchasesOfUsername(String username)</code>	PurchaseRepository	<i>QueryMethod</i>
<code>getUserSpendingMoreThan(float amount)</code>	UserRepository	<i>@Query</i>
<code>getTopNSuppliersInPurchases(int n)</code>	SupplierRepository	<i>@Query</i>

Consulta	Repositorio	Estrategia elegida
<code>getCountOfPurchasesBetweenDates(Date from, Date to)</code>	PurchaseRepository	<i>QueryMethod</i>
<code>getRoutesWithStop(Stop stop)</code>	RouteRepository	<i>QueryMethod</i>
<code>getMaxStopOfRoutes()</code>	RouteRepository	<i>@Query</i>
<code>getRoutesNotSell()</code>	RouteRepository	<i>@Query</i>
<code>getTop3RoutesWithMaxRating()</code>	RouteRepository	<i>@Query</i>
<code>getMostDemandedService()</code>	ServiceRepository	<i>@Query</i>
<code>getTourGuidesWithRating1()</code>	UserRepository	<i>@Query</i>

27. Implementar cada una de las consultas de la tabla anterior en el repositorio correspondiente, utilizando la estrategia indicada. Para las consultas que requieran paginación, utilizar Pageable como parámetro adicional.

PurchaseRepository

```
List<Purchase> findByUserUsername(String username);

long countByDateBetween(Date start, Date end);
```

UserRepository

```
@Query("""
    SELECT DISTINCT p.user FROM Purchase p
    WHERE p.totalPrice ≥ :amount
""")
List<User> findUsersSpendingMoreThan(@Param("amount") float amount);

@Query("""
    SELECT DISTINCT g FROM Route r
    JOIN r.tourGuides g, Purchase p
    WHERE p.route = r AND p.review.rating = 1
""")
List<TourGuideUser> findTourGuidesWithRating1();
```

SupplierRepository

```
@Query("""
    SELECT s.supplier FROM Service s
```

```

        JOIN s.itemServiceList item
        JOIN item.purchase p
        GROUP BY s.supplier
        ORDER BY COUNT(p) DESC
    """)
    List<Supplier> findTopSuppliersInPurchases(Pageable pageable);

```

RouteRepository

```

    List<Route> findByStopsContaining(Stop stop);

    @Query("SELECT MAX(SIZE(r.stops)) FROM Route r")
    Long findMaxStopOfRoutes();

    @Query("""
        SELECT r FROM Route r
        WHERE r NOT IN (
            SELECT DISTINCT p.route FROM Purchase p
        )
    """)
    List<Route> findRoutesNotSell();

    @Query("""
        SELECT p.route FROM Purchase p
        WHERE p.review IS NOT NULL
        GROUP BY p.route
        ORDER BY AVG(p.review.rating) DESC
    """)
    List<Route> findTop3RoutesWithMaxRating(Pageable pageable);

```

ServiceRepository

```

    @Query("""
        SELECT item.service FROM ItemService item
        GROUP BY item.service
        ORDER BY SUM(item.quantity) DESC
    """)
    List<Service> findMostDemandedService(Pageable pageable);

```

28. Comparar la implementación de al menos dos consultas complejas (por ejemplo `getTopNSuppliersInPurchases` y `getTop3RoutesWithMaxRating`) entre la Práctica 1 y esta práctica. ¿Qué diferencias hay en términos de cantidad de código, legibilidad y acoplamiento a la infraestructura de persistencia? A partir de esta comparación, ¿Qué es el

código boilerplate y que ventaja concreta representa su reducción para el mantenimiento de un proyecto real?

Característica	Práctica 1	Práctica 2
<i>Cantidad de Código</i>	Manejo explícito de <code>Session</code> Creación manual de la query Configuración manual del límite (<code>setMaxResults</code>) Más líneas de código	Declarativo (solo se define <i>qué</i> hacer, no <i>cómo</i>) Sin manejo de sesión Paginación externalizada (<code>Pageable</code>) Menos código
<i>Legibilidad</i>	Mezcla lógica de negocio + detalles técnicos Más "ruido" (session, query, configuración) Menos clara a simple vista	Query separada y limpia Código más declarativo
<i>Acoplamiento a la infraestructura</i>	Fuerte acoplamiento a: <code>SessionFactory</code> y API de Hibernate Difícil de cambiar de tecnología	Bajo acoplamiento Se trabaja contra interfaces (<code>Repository</code>) Spring abstrae la implementación

El **Código Boilerplate** es código:

- Repetitivo.
- Necesario para que algo funcione.
- Pero que no aporta lógica de negocio.

Si reducimos el **Código Boilerplate** ganamos lo siguiente:

1. *Menos errores*: Menos código repetido → menos bugs
2. *Mayor mantenibilidad*: Cambios afectan menos lugares y Código más claro
3. *Mayor productividad*: Se escribe menos código y Desarrollo más rápido
4. *Código más expresivo*: Se entiende "qué hace" sin leer detalles técnicos

Sección 3 - Transacciones

3.1 Aspectos avanzados de @Transactional

29. ¿Cuál es el atributo `readOnly = true` en `@Transactional`? ¿Qué optimizaciones activa? ¿En qué métodos del servicio conviene aplicarlo?

El `@Transactional(readOnly = true)` es una configuración de Spring que indica que la transacción es **solo de lectura**, es decir, no se van a realizar cambios en la base de datos.

Cuando se utiliza se activan las siguientes optimizaciones:

- *Desactiva el dirty checking:* Hibernate no controla cambios en las entidades ni compara el estado inicial vs. el final.
- *Evita el flush automático:* No intenta sincronizar el contexto en la base de datos, tampoco ejecuta `UPDATE`, `INSERT`, etc.
- *Optimizaciones a nivel de base de datos:* Dependiendo el driver, algunas base de datos pueden optimizar lecturas y usar transacciones más livianas.
- *Menor uso de memoria:* No se mantiene snapshots de entidades.

Métodos donde conviene aplicarlo:

- *Búsquedas simples*

```
getUserById  
getUserByUsername  
getRouteById  
getPurchaseByCode
```

- *Consultas con filtros*

```
getStopByNameStart  
getRoutesBelowPrice  
getRoutesWithStop
```

- *Consultas agregadas/estadísticas*

```
getUserSpendingMoreThan  
getTopNSuppliersInPurchases  
getCountOfPurchasesBetweenDates  
getMaxStopOfRoutes  
getRoutesNotSell  
getTop3RoutesWithMaxRating  
getMostDemandedService  
getTourGuidesWithRating1
```

30. ¿Cómo maneja Spring el rollback automático? ¿Sobre qué tipos de excepción hace rollback por defecto? ¿Cómo se configura para el rollback? Dar un ejemplo con el modelo

Spring maneja el **rollback automáticamente** mediante `@Transactional`. Cuando ocurre una excepción durante la ejecución de un método transaccional, la transacción se marca para rollback y se revierten los cambios al finalizar.

Por defecto, Spring realiza rollback solo ante **excepciones no chequeadas** (`RuntimeException` y `Error`). No realiza rollback automáticamente ante **excepciones chequeadas** como `Exception` o excepciones definidas por el usuario.

Para configurar el rollback en estos casos se utiliza el **atributo** `rollbackFor` de `@Transactional`, indicando las excepciones que deben provocar rollback.

Ejemplo:

```
@Transactional(rollbackFor = ToursException.class)
public User createUser(...) throws ToursException {
    ...
}
```

3.2 Adaptación de la capa de Servicio

31. Revisar y actualizar la capa de servicio de la Práctica 1 para que utilice los repositorios Spring Data JPA. Reemplazar todas las llamadas directas a la Session (`session.save`, `session.get`, `session.createQuery`, etc.) por las operaciones equivalentes de los repositorios. La lógica de negocio y las anotaciones `@Transactional` ya existentes no deben modificarse salvo que sea necesario por los cambios anteriores.

```
package unlp.info.bd2.services;

import java.util.Date;
import java.util.List;
import java.util.Optional;

import jakarta.persistence.EntityManager;

import org.springframework.data.domain.PageRequest;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.data.domain.Pageable;

import unlp.info.bd2.model.*;
import unlp.info.bd2.repositories.ItemServiceRepository;
import unlp.info.bd2.repositories.PurchaseRepository;
import unlp.info.bd2.repositories.ReviewRepository;
import unlp.info.bd2.repositories.RouteRepository;
```

```
import unlp.info.bd2.repositories.ServiceRepository;
import unlp.info.bd2.repositories.StopRepository;
import unlp.info.bd2.repositories.SupplierRepository;
import unlp.info.bd2.repositories.UserRepository;
import unlp.info.bd2.utils.ToursException;

@Service
@Transactional
public class ToursServiceImpl implements ToursService {

    private final UserRepository userRepository;

    private final RouteRepository routeRepository;

    private final PurchaseRepository purchaseRepository;

    private final ServiceRepository serviceRepository;

    private final SupplierRepository supplierRepository;

    private final ReviewRepository reviewRepository;

    private final StopRepository stopRepository;

    private final ItemServiceRepository itemServiceRepository;

    private final EntityManager entityManager;

    public ToursServiceImpl(PurchaseRepository purchaseRepository,
        ReviewRepository reviewRepository,
        RouteRepository routeRepository,
        ServiceRepository serviceRepository,
        SupplierRepository supplierRepository,
        UserRepository userRepository,
        StopRepository stopRepository,
        ItemServiceRepository itemServiceRepository,
        EntityManager entityManager) {
        this.purchaseRepository = purchaseRepository;
        this.reviewRepository = reviewRepository;
        this.routeRepository = routeRepository;
        this.serviceRepository = serviceRepository;
        this.supplierRepository = supplierRepository;
        this.userRepository = userRepository;
        this.stopRepository = stopRepository;
        this.itemServiceRepository = itemServiceRepository;
        this.entityManager = entityManager;
    }
}
```

```

}

// — USER —————

@Override
public User createUser(String username, String password, String
fullName,
    String email, Date birthdate, String phoneNumber) throws
ToursException {
    try {
        User user = new User();
        user.setUsername(username);
        user.setPassword(password);
        user.setName(fullName);
        user.setEmail(email);
        user.setBirthdate(birthdate);
        user.setPhoneNumber(phoneNumber);
        user.setActive(true);
        User saved = userRepository.save(user);
        // Forzamos flush para que el constraint unique se evalúe dentro
de esta transacción
        entityManager.flush();
        return saved;
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

@Override
public DriverUser createDriverUser(String username, String password,
    String fullName, String email, Date birthdate,
    String phoneNumber, String expedient) throws ToursException {
    try {
        DriverUser driver = new DriverUser();
        driver.setUsername(username);
        driver.setPassword(password);
        driver.setName(fullName);
        driver.setEmail(email);
        driver.setBirthdate(birthdate);
        driver.setPhoneNumber(phoneNumber);
        driver.setExpedient(expedient);
        driver.setActive(true);
        DriverUser saved = (DriverUser) userRepository.save(driver);
        entityManager.flush();
        return saved;
    } catch (Exception e) {

```

```

        throw new ToursException("Constraint Violation");
    }
}

@Override
public TourGuideUser createTourGuideUser(String username, String
password,
    String fullName, String email, Date birthdate,
    String phoneNumber, String education) throws ToursException {
    try {
        TourGuideUser guide = new TourGuideUser();
        guide.setUsername(username);
        guide.setPassword(password);
        guide.setName(fullName);
        guide.setEmail(email);
        guide.setBirthdate(birthdate);
        guide.setPhoneNumber(phoneNumber);
        guide.setEducation(education);
        guide.setActive(true);
        TourGuideUser saved = (TourGuideUser)
userRepository.save(guide);
        entityManager.flush();
        return saved;
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

@Override
@Transactional(readOnly = true)
public Optional<User> getUserById(Long id) throws ToursException {
    return userRepository.findById(id);
}

@Override
@Transactional(readOnly = true)
public Optional<User> getUserByUsername(String username) throws
ToursException {
    return userRepository.findByUsername(username);
}

@Override
public User updateUser(User user) throws ToursException {
    if (!userRepository.existsById(user.getId())) {
        throw new ToursException("Usuario no encontrado");
    }
}

```

```

        return userRepository.save(user);
    }

    @Override
    public void deleteUser(User user) throws ToursException {
        if (!user.isActive()) {
            throw new ToursException("El usuario se encuentra desactivado");
        }
        if (user instanceof TourGuideUser) {
            Long count = routeRepository.countRoutesByGuide(user.getId());
            if (count > 0) {
                throw new ToursException("El usuario no puede ser
desactivado");
            }
        }
        if (user.getPurchaseList() != null &&
!user.getPurchaseList().isEmpty()) {
            user.setActive(false);
        } else {
            userRepository.delete(user);
        }
    }

    // ——— STOP —————

    @Override
    public Stop createStop(String name, String description) throws
ToursException {
        Stop stop = new Stop();
        stop.setName(name);
        stop.setDescription(description);
        return stopRepository.save(stop);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Stop> getStopByNameStart(String name) {
        return stopRepository.findByNameStartingWith(name);
    }

    // ——— ROUTE —————

    @Override
    public Route createRoute(String name, float price, float totalKm,
        int maxNumberOfUsers, List<Stop> stops) throws ToursException {
        Route route = new Route();

```

```

        route.setName(name);
        route.setPrice(price);
        route.setTotalKm(totalKm);
        route.setMaxNumberUsers(maxNumberOfUsers);
        route.setStops(stops);
        return routeRepository.save(route);
    }

    @Override
    @Transactional(readOnly = true)
    public Optional<Route> getRouteById(Long id) {
        return routeRepository.findById(id);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Route> getRoutesBelowPrice(float price) {
        return routeRepository.findByPriceLessThan(price);
    }

    @Override
    public void assignDriverByUsername(String username, Long idRoute) throws
    ToursException {
        Optional<User> optUser = userRepository.findByUsername(username);
        if (optUser.isEmpty() || !(optUser.get() instanceof DriverUser)) {
            throw new ToursException("No pudo realizarse la asignación");
        }
        Optional<Route> optRoute = routeRepository.findById(idRoute);
        if (optRoute.isEmpty()) {
            throw new ToursException("No pudo realizarse la asignación");
        }
        Route route = optRoute.get();
        route.getDriverList().add((DriverUser) optUser.get());
    }

    @Override
    public void assignTourGuideByUsername(String username, Long idRoute)
    throws ToursException {
        Optional<User> optUser = userRepository.findByUsername(username);
        if (optUser.isEmpty() || !(optUser.get() instanceof TourGuideUser))
    {
        throw new ToursException("No pudo realizarse la asignación");
    }
        Optional<Route> optRoute = routeRepository.findById(idRoute);
        if (optRoute.isEmpty()) {
            throw new ToursException("No pudo realizarse la asignación");
        }
    }

```

```

    }
    Route route = optRoute.get();
    route.getTourGuideList().add((TourGuideUser) optUser.get());
}

// — SUPPLIER —————

@Override
public Supplier createSupplier(String businessName,
    String authorizationNumber) throws ToursException {
    try {
        Supplier supplier = new Supplier();
        supplier.setBusinessName(businessName);
        supplier.setAuthorizationNumber(authorizationNumber);
        Supplier saved = supplierRepository.save(supplier);
        entityManager.flush();
        return saved;
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

@Override
@Transactional(readOnly = true)
public Optional<Supplier> getSupplierById(Long id) {
    return supplierRepository.findById(id);
}

@Override
@Transactional(readOnly = true)
public Optional<Supplier> getSupplierByAuthorizationNumber(String
authorizationNumber) {
    return
supplierRepository.findByAuthorizationNumber(authorizationNumber);
}

// — SERVICE —————

@Override
public unlp.info.bd2.model.Service addServiceToSupplier(String name,
float price,
    String description, Supplier supplier) throws ToursException {
    try {
        unlp.info.bd2.model.Service service = new
unlp.info.bd2.model.Service();
        service.setName(name);
    }
}

```

```

        service.setPrice(price);
        service.setDescription(description);
        service.setSupplier(supplier);
        supplier.getServices().add(service);
        unlp.info.bd2.model.Service saved =
serviceRepository.save(service);
        // FIX: flush para detectar constraint de nombre duplicado
dentro de la transacción
        entityManager.flush();
        return saved;
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

@Override
public unlp.info.bd2.model.Service updateServicePriceById(Long id, float
newPrice) throws ToursException {
    unlp.info.bd2.model.Service service = serviceRepository.findById(id)
        .orElseThrow(() → new ToursException("No existe el producto"));

    service.setPrice(newPrice);
    return service;
}

@Override
@Transactional(readOnly = true)
public Optional<unlp.info.bd2.model.Service>
getServiceByNameAndSupplierId(
    String name, Long id) throws ToursException {
    return serviceRepository.findByNameAndSupplierId(name, id);
}

// ——— PURCHASE ———

@Override
public Purchase createPurchase(String code, Route route,
    User user) throws ToursException {
    return createPurchase(code, new Date(), route, user);
}

@Override
public Purchase createPurchase(String code, Date date, Route route,
    User user) throws ToursException {
    long count = routeRepository.countPurchasesByRoute(route.getId());
    if (count ≥ route.getMaxNumberUsers()) {

```

```

        throw new ToursException("No puede realizarse la compra");
    }
    try {
        Purchase purchase = new Purchase();
        purchase.setCode(code);
        purchase.setDate(date);
        purchase.setRoute(route);
        purchase.setUser(user);
        purchase.setTotalPrice(route.getPrice());
        user.getPurchaseList().add(purchase);
        Purchase saved = purchaseRepository.save(purchase);
        entityManager.flush();
        return saved;
    } catch (Exception e) {
        throw new ToursException("Constraint Violation");
    }
}

@Override
public ItemService addItemToPurchase(unlp.info.bd2.model.Service
service, int quantity,
    Purchase purchase) throws ToursException {
    ItemService item = new ItemService();
    item.setService(service);
    item.setQuantity(quantity);
    // FIX: Purchase.addItem() ya suma al totalPrice internamente,
    // por eso NO sumamos manualmente acá – evitamos el precio
    duplicado.
    purchase.addItem(item);
    service.getItemServiceList().add(item);
    return itemServiceRepository.save(item);
}

@Override
@Transactional(readOnly = true)
public Optional<Purchase> getPurchaseByCode(String code) {
    return purchaseRepository.findByCode(code);
}

@Override
public void deletePurchase(Purchase purchase) throws ToursException {
    if (purchase.getUser() != null) {
        purchase.getUser().getPurchaseList().remove(purchase);
    }
    purchaseRepository.delete(purchase);
    entityManager.flush();
}

```

```

}

// — REVIEW —————

@Override
public Review addReviewToPurchase(int rating, String comment,
    Purchase purchase) throws ToursException {
    Review review = new Review();
    review.setRating(rating);
    review.setComment(comment);
    review.setPurchase(purchase);
    purchase.setReview(review);
    return reviewRepository.save(review);
}

// — HQL —————

@Override
@Transactional(readOnly = true)
public List<Purchase> getAllPurchasesOfUsername(String username) {
    return purchaseRepository.findByUserUsername(username);
}

@Override
@Transactional(readOnly = true)
public List<User> getUserSpendingMoreThan(float mount) {
    return userRepository.findUsersSpendingMoreThan(mount);
}

@Override
@Transactional(readOnly = true)
public List<Supplier> getTopNSuppliersInPurchases(int n) {
    Pageable pageable = PageRequest.of(0, n);
    return supplierRepository.findTopSuppliersInPurchases(pageable);
}

@Override
@Transactional(readOnly = true)
public Long getCountOfPurchasesBetweenDates(Date start, Date end) {
    return purchaseRepository.countByDateBetween(start, end);
}

@Override
@Transactional(readOnly = true)
public List<Route> getRoutesWithStop(Stop stop) {
    return routeRepository.findByStopsContaining(stop);
}

```

```

    }

    @Override
    @Transactional(readOnly = true)
    public Long getMaxStopOfRoutes() {
        return routeRepository.findMaxStopOfRoutes();
    }

    @Override
    @Transactional(readOnly = true)
    public List<Route> getRoutesNotSell() {
        return routeRepository.findRoutesNotSell();
    }

    @Override
    @Transactional(readOnly = true)
    public List<Route> getTop3RoutesWithMaxRating() {
        Pageable pageable = PageRequest.of(0, 3);
        return routeRepository.findTop3RoutesWithMaxRating(pageable);
    }

    @Override
    @Transactional(readOnly = true)
    public unlp.info.bd2.model.Service getMostDemandedService() {
        Pageable pageable = PageRequest.of(0, 1);
        return serviceRepository.findMostDemandedService(pageable)
            .stream()
            .findFirst()
            .orElse(null);
    }

    @Override
    @Transactional(readOnly = true)
    public List<TourGuideUser> getTourGuidesWithRating1() {
        return userRepository.findTourGuidesWithRating1();
    }
}

```

Sección 4 - DTOs y Proyecciones

4.1 Qué es un DTO y cuándo usarlo

32. ¿Qué es un DTO (Data Transfer Object)? ¿Cuál es su propósito dentro de la arquitectura de una aplicación? ¿Por qué no siempre conviene devolver una entidad JPA directamente desde la capa de acceso a datos?

Un **DTO (Data Transfer Object)** es un objeto diseñado exclusivamente para **transportar datos** entre diferentes procesos o capas de una aplicación. Según la definición de Martin Fowler, su fin primordial es reducir el número de llamadas a métodos entre procesos. Se distingue de las entidades en que es un objeto "plano" que **no contiene lógica de negocio**, limitándose únicamente a poseer atributos y sus correspondientes métodos *getters* y *setters*

Propósito dentro de la arquitectura: El propósito de un DTO es actuar como un contenedor de información volátil que facilita la comunicación entre capas. Sus funciones principales incluyen:

- **Separación de capas:** Refuerza la independencia entre el modelo de dominio y la interfaz de usuario, permitiendo que esta última reciba solo lo que necesita sin conocer la estructura interna de la base de datos.
- **Adaptación de datos:** Permite cambiar el formato de la información, combinar atributos de múltiples entidades o agregar campos calculados que no existen en el modelo original.
- **Optimización:** Permite enviar en una sola estructura toda la información requerida para un caso de uso específico, evitando múltiples consultas.

¿Por qué no devolver entidades JPA directamente?

- **Restricciones transaccionales:** No es posible leer información de una entidad (especialmente sus asociaciones configuradas como `LAZY`) fuera del contexto de una **transacción activa**. Si la transacción se cierra en la capa de servicio y la capa web intenta acceder a un colaborador de la entidad, se producirá una excepción técnica (como la `LazyInitializationException`).
- **Seguridad y Encapsulamiento:** Las entidades contienen **lógica de negocio** y un protocolo público que el cliente o la interfaz de usuario no deberían poder invocar indebidamente. Devolver un DTO asegura que solo se expongan datos y no comportamiento.
- **Acoplamiento:** Exponer entidades acopla fuertemente la interfaz con el modelo de datos. Cualquier cambio en la estructura de las tablas obligaría a modificar la capa de presentación.
- **Riesgos de Serialización:** Las entidades suelen tener relaciones bidireccionales que pueden generar **ciclos de serialización** infinitos al intentar convertirlas a formatos como JSON para un endpoint.
- **Gestión de Concurrencia:** Mantener entidades en memoria durante mucho tiempo (mientras el usuario interactúa con la interfaz) incrementa el riesgo de **conflictos de versiones** y datos obsoletos al intentar persistir cambios. Los DTOs, al ser copias desconectadas, eliminan este vínculo directo durante la visualización.

33. Describir dos situaciones concretas del modelo donde devolver un DTO sería más adecuado que devolver la entidad completa. Para cada caso indicar qué campos contendría el DTO y por qué.

Situaciones concretas:

- *Consulta de usuario para lectura pública*
 - Situación: métodos como `getUserById` o `getUserByUsername` hoy devuelven `User` completo.
 - Problema: `User` incluye campos sensibles y relaciones que no siempre deberían salir, como `password` y `purchaseList` en `User`.
 - DTO propuesto: `UserProfileDTO`
 - Campos: `id`, `username`, `name`, `email`, `phoneNumber`, `active`, `userType`.
 - Por qué:
 - Evita exponer `password`.
 - Evita acoplar el contrato API a la estructura interna de JPA.
 - Reduce riesgo de cargar relaciones innecesarias (`purchaseList`) y de serialización recursiva.
- *Resumen de compras de un usuario*
 - Situación: `getAllPurchasesOfUsername` devuelve `List<Purchase>`
 - Problema: `Purchase` arrastra un grafo grande: `user`, `route`, `review`, `itemServiceList` en `Purchase`. Para una grilla/listado normalmente no hace falta todo eso.
 - DTO propuesto: `PurchaseSummaryDTO`
 - Campos: `code`, `date`, `totalPrice`, `routeId`, `routeName`, `hasReview`, `reviewRating`, `itemsCount`.
 - Por qué:
 - Devuelve exactamente lo que una pantalla de historial necesita.
 - Evita traer y serializar entidades completas relacionadas.
 - Mejora performance y claridad del contrato (menos payload, menos dependencias a detalles internos de `Route/User/Review`).

34. ¿Qué riesgos concretos tiene exponer entidades JPA directamente como respuesta de un servicio o endpoint? Mencionar al menos: ciclos de serialización y acoplamiento entre capas

Riesgos Principales

- **Ciclos de Serialización:** Las entidades JPA suelen tener relaciones bidireccionales (por ejemplo, una `Purchase` referencia a un `User`, y el `User` tiene una lista de `Purchases`). Al intentar convertir estos objetos a un formato como JSON, el serializador puede entrar en un

bucle infinito al navegar de un objeto a otro de forma recursiva, lo que provoca fallos en el sistema o errores de desbordamiento de memoria.

- **Acoplamiento fuerte entre capas:** Al devolver la entidad, se expone la estructura física de la base de datos directamente a la capa de presentación o a clientes externos. Esto implica que cualquier cambio en el esquema de la base de datos (como renombrar una columna) impactará automáticamente en la API, rompiendo la compatibilidad con los clientes que la consumen.
- **Exposición de datos sensibles y lógica interna:** Las entidades suelen contener información que no debe ser pública (como contraseñas o datos técnicos) y poseen **lógica de negocio** propia del modelo de dominio. Al entregar la entidad, el cliente podría acceder a datos privados o intentar invocar métodos de comportamiento que deberían estar restringidos a la capa de servicio.
- **Problemas de carga perezosa (LazyInitializationException):** El modelo utiliza frecuentemente el `FetchType.LAZY` para optimizar el acceso a los datos. Si el endpoint intenta acceder a una colección o relación no cargada una vez que la **transacción se ha cerrado** en la capa de servicio, se producirá una excepción técnica, ya que no se permite leer información de la base de datos fuera de un marco transaccional activo.

4.2 Implementación

35. Definir e implementar un DTO que resuma información de las rutas del modelo: nombre de la ruta, cantidad de compras realizadas y el precio promedio de esas compras. Implementar la consulta correspondiente en `RouteRepository`.

RoutePurchaseSummaryDTO

```
package unlp.info.bd2.dto;

public class RoutePurchaseSummaryDTO {

    private final String routeName;
    private final long purchaseCount;
    private final double averagePurchasePrice;

    public RoutePurchaseSummaryDTO(String routeName, long purchaseCount,
double averagePurchasePrice) {
        this.routeName = routeName;
        this.purchaseCount = purchaseCount;
        this.averagePurchasePrice = averagePurchasePrice;
    }

    public String getRouteName() {
        return routeName;
    }
}
```

```

    }

    public long getPurchaseCount() {
        return purchaseCount;
    }

    public double getAveragePurchasePrice() {
        return averagePurchasePrice;
    }
}

```

RouteRepository

```

@Query("""
    SELECT new unlp.info.bd2.dto.RoutePurchaseSummaryDTO(
        r.name,
        COUNT(p),
        COALESCE(AVG(p.totalPrice), 0)
    )
    FROM Route r
    LEFT JOIN Purchase p ON p.route = r
    GROUP BY r.id, r.name
    ORDER BY r.name
""")
List<RoutePurchaseSummaryDTO> findRoutePurchaseSummaries();

```

ToursService

```

List<RoutePurchaseSummaryDTO> getRoutePurchaseSummaries();

```

ToursServiceImpl

```

@Override
@Transactional(readOnly = true)
public List<RoutePurchaseSummaryDTO> getRoutePurchaseSummaries() {
    return routeRepository.findRoutePurchaseSummaries();
}

```

Sección 5 - Borrado Lógico (Soft Delete)

5.1 Concepto y estrategias

36. ¿Que es el borrado lógico (soft delete)? Pensar en el modelo de tours: si un usuario decide darse de baja del sistema, ¿tiene sentido eliminar físicamente su registro? ¿Qué ocurriría con todas las compras que ese usuario ha realizado? Describir por qué el borrado lógico resuelve este problema y qué ventaja ofrece frente al borrado físico en este caso concreto.

El **borrado lógico (soft delete)** es una estrategia de persistencia que consiste en evitar la eliminación física de un registro de la base de datos, optando en su lugar por marcarlo con un estado de **inactivo o borrado**. Generalmente, esto se logra mediante un campo booleano (ej. `deleted = true`) o un campo de fecha (`deletedAt`), permitiendo que el dato permanezca en el soporte físico aunque sea invisible para la lógica ordinaria de la aplicación.

En el contexto de nuestro modelo:

- **¿Tiene sentido eliminar físicamente al usuario?** No, no tiene sentido desde el punto de vista de la integridad y el negocio. Si un usuario se da de baja, su registro físico es necesario para mantener la **coherencia histórica** de la base de datos.
- **¿Qué ocurriría con sus compras?** Si se realizara un borrado físico, todas las instancias de `Purchase` asociadas a ese usuario quedarían **huérfanas** o violarían las restricciones de integridad referencial (Foreign Keys) del motor de base de datos. Esto invalidaría reportes financieros, estadísticas de rutas y el seguimiento de los servicios prestados.
- **¿Cómo resuelve esto el borrado lógico?** El borrado lógico permite que el registro del `User` siga existiendo en la tabla física, manteniendo así las **claves foráneas** de las compras apuntando a un registro válido. Al mismo tiempo, mediante herramientas como la anotación `@Where` en Hibernate, la aplicación puede ignorar automáticamente a estos usuarios en las búsquedas cotidianas (como en el login o listados de usuarios activos), simulando que ya no están en el sistema.

Ventajas frente al borrado físico en este caso:

- **Preservación de la Integridad Referencial:** Evita inconsistencias en la base de datos al no romper los vínculos con entidades dependientes como `Purchase` o `Review`.
- **Mantenimiento de Historial y Auditoría:** Permite que la empresa conserve la información de las transacciones pasadas para análisis estadísticos y contables, incluso si el cliente ya no utiliza el servicio.
- **Recuperabilidad:** Facilita la reactivación de la cuenta en caso de que el usuario decida volver al sistema, simplemente revirtiendo el estado del marcador lógico.
- **Seguridad en el manejo de datos:** El método `canBeDeactivate()` en el modelo busca precisamente evitar que se eliminen usuarios con compras o asignaciones vigentes; el borrado lógico permite "desactivar" al usuario cumpliendo con esta restricción de seguridad sin perder los datos vinculados.

37. Describir dos estrategias para implementar soft delete en JPA/Hibernate: campo booleano (active/deleted) y campo de fecha (deletedAt). ¿Qué diferencias hay entre ambas en cuanto a consultas y recuperación de datos?

Estrategia de Campo Booleano (`active` / `deleted`): Es la forma más simple y común de implementación.

- **Mecanismo:** Se agrega un atributo booleano a la entidad (por ejemplo, `private boolean deleted = false`).
- **Implementación:** Se utiliza la anotación `@SQLDelete` para que, al invocar el método `delete()`, Hibernate ejecute un `UPDATE` cambiando el valor del booleano a `true` en lugar de un `DELETE` físico.
- **Filtrado:** Se combina con `@Where(clause = "deleted = false")` para que todas las consultas ordinarias (`findAll`, `findById`) ignoren automáticamente los registros marcados como borrados.

Estrategia de Campo de Fecha (`deletedAt`): Esta estrategia es más avanzada y orientada a la auditoría.

- **Mecanismo:** Se utiliza un campo de tipo fecha/tiempo (ej. `LocalDateTime deletedAt`). Si el objeto está activo, el campo es `null`; si está borrado, contiene la fecha exacta de la eliminación.
- **Implementación:** El `@SQLDelete` realiza un `UPDATE` asignando la fecha actual al campo `deleted_at`.
- **Filtrado:** La cláusula global sería `@Where(clause = "deleted_at IS NULL")`.

Diferencias en Consultas y Recuperación de Datos

Característica	Campo Booleano	Campo de Fecha (<code>deletedAt</code>)
Simplicidad de Filtro	Muy alta. Solo se chequea un estado verdadero/falso.	Media. Requiere chequear la nulidad del campo.
Información de Auditoría	Baja. Solo se sabe que el registro fue borrado, pero no cuándo.	Alta. Permite saber el momento exacto de la baja para reportes históricos.
Consultas Temporales	No es posible realizarlas con este campo.	Permite responder preguntas como "¿Cuántos usuarios se dieron de baja el mes pasado?".
Recuperación (Un-delete)	Se cambia el flag a <code>false</code> .	Se vuelve a poner el campo en <code>null</code> .
Impacto en <code>@Where</code>	Ambas se ven afectadas por igual: <code>@Where</code> filtra los datos	Ambas se ven afectadas por igual: <code>@Where</code> filtra los datos

Característica	Campo Booleano	Campo de Fecha (<code>deletedAt</code>)
	globalmente para el ORM.	globalmente para el ORM.

38. ¿Qué es la anotación `@SQLDelete` de JPA? ¿Qué permite hacer? ¿Cómo se combina con `@Where` para que las consultas ordinarias ignoren los registros borrados? Mostrar un ejemplo de uso sobre la entidad `User`.

Es una anotación que permite **sobrescribir el comando SQL de eliminación** predeterminado que Hibernate genera para una entidad.

- **Redefinición del comportamiento:** En lugar de ejecutar una sentencia física `DELETE`, permite que al invocar los métodos de borrado (como `delete()` o `deleteById()`), el sistema ejecute una sentencia **UPDATE**.
- **Marcado de registros:** Esta sentencia `UPDATE` cambia el estado de un campo específico (generalmente un booleano como `deleted` o `active`) para marcar el registro como inactivo en lugar de quitarlo físicamente de la tabla.

Cómo se combina con `@Where`: Mientras que `@SQLDelete` se encarga de la escritura (cómo se "borra"), la anotación **`@Where`** se encarga de la lectura (qué se muestra).

- **Filtro Global:** `@Where` permite definir una cláusula SQL (por ejemplo, `clause = "deleted = false"`) que Hibernate aplicará de forma **automática y global** a todas las consultas generadas para esa entidad.
- **Transparencia:** Gracias a este filtro, las consultas ordinarias como `findAll()`, `findById()` o los *Query Methods* del repositorio ignorarán automáticamente los registros que tengan la marca de borrado, como si realmente no existieran en la base de datos.

Ejemplo conceptual en la entidad `User`

```
import org.hibernate.annotations.SQLDelete;
import org.hibernate.annotations.Where;
import jakarta.persistence.*;

@Entity
@Table(name = "users")
@SQLDelete(sql = "UPDATE users SET active = false WHERE id = ?")
@Where(clause = "active = true")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"user_seq")
    @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence",
```

```

allocationSize = 50)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private boolean active = true;

    // getters y setters
}

```

5.2 Implementación en el modelo

39. Implementar soft delete para la entidad User usando @SQLDelete y @Where. La implementación debe:

1. Agregar el campo necesario a la entidad User (campo y anotaciones).
2. Redefinir el comportamiento de delete para que marque el registro en lugar de eliminarlo.
3. Asegurar que todas las consultas ordinarias (findAll, findById, Query Methods) excluyan automáticamente los usuarios borrados.

User

```

@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "user_type", discriminatorType =
DiscriminatorType.STRING)
@SQLDelete(sql = "UPDATE users SET active = false WHERE id = ?")
@Where(clause = "active = true")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"user_seq")
    @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence",
allocationSize = 50)
    private Long id;

    @Column(nullable = false, unique = true)

```

```

private String username;

@Column(nullable = false)
private String password;

@Column(nullable = false)
private String name;

@Column
private String email;

@Column
private Date birthdate;

@Column
private String phoneNumber;

@Column(nullable = false)
private boolean active = true;

@OneToMany(
    mappedBy = "user",
    fetch = FetchType.LAZY,
    cascade = {
        CascadeType.PERSIST,
        CascadeType.MERGE
    })
private List<Purchase> purchaseList = new ArrayList<>();

//getters y setters existentes...

```

ToursApplicationTests: Se tuvo que modificar `deleteUserTest` para que un usuario dado de baja no aparezca en búsquedas ordinarias.

```

@Test
void deleteUserTest() throws ToursException {
    User user1 = this.toursService.createUser("user1", "1234", "Usuario
Uno", "user1@gmail.com", dob1, "000111222333");

    assertTrue(user1.isActive());
    this.toursService.deleteUser(user1);
    assertTrue(this.toursService.getUserByUsername("user1").isEmpty());

    User user2 = this.toursService.createUser("user2", "1234", "Usuario
Dos", "user2@gmail.com", dob2, "000111222334");

```

```

        Stop stop1 = this.toursService.createStop("Estadio Monumental", "Estadio
de River Plate");
        Stop stop2 = this.toursService.createStop("Estadio La Bombonera",
"Estadio de Boca Junions");
        Stop stop3 = this.toursService.createStop("Estadio Libertadores de
America", "Estadio de Independiente");
        List<Stop> stops1 = new ArrayList<Stop>(Arrays.asList(stop1, stop2,
stop3));
        Route route1 = this.toursService.createRoute("Estadios", 20000, 55.5f,
2, stops1);
        Purchase purchase1 = this.toursService.createPurchase("100", dyes,
route1, user2);
        assertTrue(user2.isActive());
        this.toursService.deleteUser(user2);
        Optional<User> optionalUser2 =
this.toursService.getUserByUsername("user2");
        assertTrue(optionalUser2.isEmpty());
        assertFalse(user2.isActive());

        assertThrows(ToursException.class, () →
this.toursService.deleteUser(user2), "El usuario se encuentra desactivado");

        TourGuideUser tourGuideUser =
this.toursService.createTourGuideUser("user6", "1234|", "Usuario TourGuide",
"user6@gmail.com", dob2, "000111222555", "edu...");
        this.toursService.assignTourGuideByUsername(tourGuideUser.getUsername(),
route1.getId());
        assertTrue(tourGuideUser.isActive());
        assertThrows(ToursException.class, () →
this.toursService.deleteUser(tourGuideUser), "El usuario no puede ser
desactivado");
    }

```

40. La implementación actual muestra el método `canBeDeactivate()` en `User`, `DriverUser` y `TourGuideUser`. El objetivo de estos métodos es evitar que se eliminen usuarios que poseen compras o asignaciones y, por tanto queden inconsistencias en la base de datos. Describa como esta nueva implementación afecta dichos métodos.

En las clases mencionadas no existe ese método. La lógica real hoy está en `ToursServiceImpl`: primero bloquea la baja de un `TourGuideUser` si tiene rutas asignadas, y si el usuario tiene compras no lo elimina físicamente, sino que lo desactiva. Con el **soft delete** agregado en `User`, esa desactivación se vuelve además coherente a nivel ORM: `@SQLDelete` transforma el borrado en un `UPDATE active = false` y `@Where(clause = "active = true")` hace que las consultas ordinarias ignoren a los usuarios inactivos.

Entonces, si el enunciado menciona `canBeDeactivate()` , en este modelo actual esa responsabilidad quedó repartida así:

- La validación de si un guía puede darse de baja está en `ToursServiceImpl` .
- La exclusión automática de usuarios borrados está en `User` .
- No hace falta un método específico `canBeDeactivate()` en las entidades para que el soft delete funcione.

En resumen, la nueva implementación hace que esos métodos sean innecesarios si lo único que querían era evitar borrado físico e inconsistencias. Lo que sí sigue siendo necesario es la regla de negocio que impide desactivar a un `TourGuideUser` con rutas asignadas, porque eso no lo resuelve `@SQLDelete/@Where` por sí solo.